Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

NEXUS - AI Powered Distributed Data Processing and Computation Orchestration Platform

Team Name: *Team #*	*OSDBMS-V-2026-T099*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Pareek, Raghvander – 240211322* *240211322@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Kathait, Soyam – 240211683* *240211683@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Kumar Singh, Tushar – 240211961* *240211322@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Kumar Singh, Tanishk – 240211960* *240211960@geu.ac.in* 

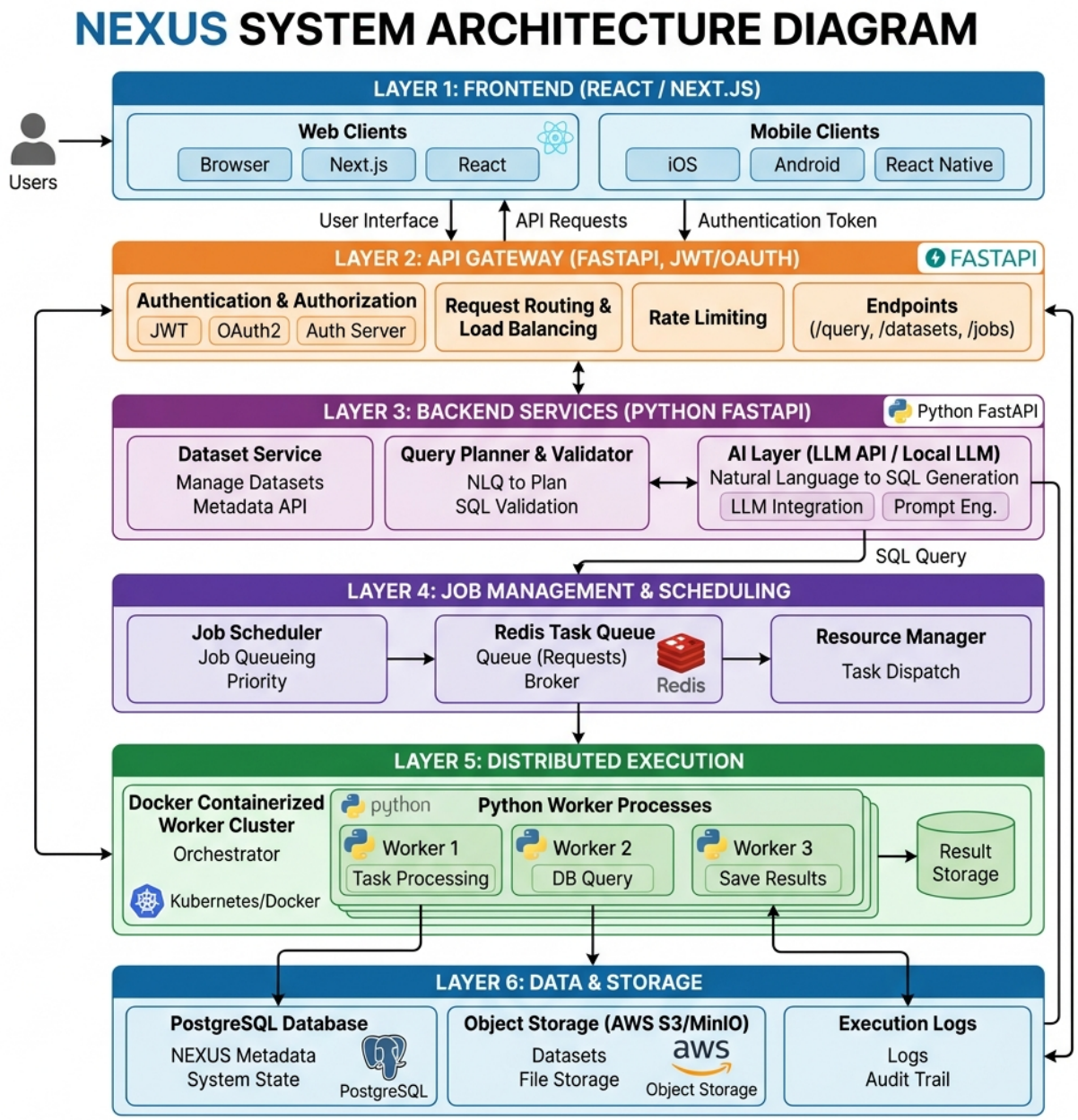
PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

NEXUS is a web-based platform designed to simplify data analysis by allowing users to interact with databases using natural language instead of writing SQL queries. It integrates Artificial Intelligence, Database Management Systems, Operating Systems, and Distributed Systems into a single application.
Users can upload structured datasets, such as CSV files, and ask analytical questions in plain English. An AI-powered Natural Language Processing module converts these questions into SQL queries. The generated queries are validated for correctness, security, and permissions before execution, then optimized and submitted to a job scheduler. The scheduler distributes workloads across worker processes according to scheduling algorithms and resource availability, supporting concurrent tasks and efficient CPU and memory utilization.
The platform also provides authentication, dataset management, execution monitoring, query history, AI-generated explanations, and visual representations of results. NEXUS aims to make advanced data analysis accessible to technical and non-technical users while supporting secure, efficient, and scalable data-driven decision-making.

Updated Project Approach and Architecture

NEXUS follows a layered client-server architecture in which each component has a defined responsibility and communicates through APIs.
Frontend: React/Next.js provides the interface for uploading datasets, submitting natural-language questions, and viewing results.
API Gateway and Backend: FastAPI is used to handle requests from the frontend and manage authentication, request routing, and rate limiting. The backend services are responsible for dataset management, query planning, and validation. We plan to integrate the Gemini API to convert users' natural-language questions into SQL queries.
Job Management and Execution: Once a query passes validation, it is forwarded to the job-management component for execution. This component includes a job scheduler, a Redis task queue, and a resource manager. Docker-based Python workers are designed to execute database operations, process assigned tasks, and store the results.
Data Layer: PostgreSQL is used to store metadata and system information. Uploaded datasets are maintained in object storage, and execution logs are recorded to help monitor operations and investigate errors.
Communication: REST APIs enable communication between the frontend and backend services. Redis manages the task queue between backend services and worker processes. We have organized the system into separate modules so that each component can be developed, tested, and maintained independently, making future changes easier.


NEXUS SYSTEM ARCHITECTURE DIAGRAM
Users
LAYER 1: FRONTEND (REACT / NEXT.JS)
Web Clients
Browser
Next.js
React
Mobile Clients
iOS
Android
React Native
User Interface
API Requests
Authentication Token
LAYER 2: API GATEWAY (FASTAPI, JWT/OAUTH)
FASTAPI
Authentication & Authorization
JWT
OAuth2
Auth Server
Request Routing & Load Balancing
Rate Limiting
Endpoints
(/query, /datasets, /jobs)
LAYER 3: BACKEND SERVICES (PYTHON FASTAPI)
Python FastAPI
Dataset Service
Manage Datasets
Metadata API
Query Planner & Validator
NLQ to Plan
SQL Validation
AI Layer (LLM API / Local LLM)
Natural Language to SQL Generation
LLM Integration
Prompt Eng.
SQL Query
LAYER 4: JOB MANAGEMENT & SCHEDULING
Job Scheduler
Job Queueing
Priority
Redis Task Queue
Queue (Requests)
Broker
Redis
Resource Manager
Task Dispatch
LAYER 5: DISTRIBUTED EXECUTION
Docker Containerized Worker Cluster
Orchestrator
Kubernetes/Docker
python
Python Worker Processes
Worker 1
Task Processing
Worker 2
DB Query
Worker 3
Save Results
Result Storage
LAYER 6: DATA & STORAGE
PostgreSQL Database
NEXUS Metadata
System State
PostgreSQL
Object Storage (AWS S3/MinIO)
Datasets
File Storage
aws
Object Storage
Execution Logs
Logs
Audit Trail

Tasks Completed

Main tasks assigned and completed during this phase.

Task Completed	Team Member
Requirement analysis, SRS preparation, and problem-statement documentation	Soyam Raj Kathait
Database schema design, PostgreSQL setup, and metadata planning	Raghvander Pareek
Frontend UI wireframes, React/Next.js project setup, and basic pages	Tanishk Kumar Singh
Backend FastAPI project setup, REST API structure, and API integration planning	Tushar Kumar Singh
AI workflow design, SQL-generation pipeline planning, and overall system architecture diagram	All members

Challenges/Roadblocks

One of the main challenges was deciding how the different parts of NEXUS would work together. We addressed this during the design stage by assigning separate roles to the frontend, backend, AI component, and database. Another area that needed attention was the PostgreSQL database design, particularly how dataset details, metadata, and execution logs would be stored. We also considered security while planning the connection between the AI component and the database. Generated SQL queries will need to be checked before they are allowed to run.
Since the project uses React/Next.js, FastAPI, PostgreSQL, Redis, and the Gemini API, connecting the different technologies is an important part of the implementation. Our plan is to build and test the components one by one, then connect them gradually. We will also check the generated queries and evaluate the system's performance and reliability during testing. Complete integration and final validation are still pending.

Tasks Pending

Task Pending	Team Member(s) to Complete
Finalize project documentation, SRS updates, and testing documentation	Soyam Raj Kathait
Implement database optimization, indexing, backup, and query-performance tuning	Raghvander Pareek
Complete frontend integration, dashboard, data visualization, and user-experience improvements	Tanishk Kumar Singh
Complete backend API integration, AI-powered SQL generation, job scheduling, and final deployment	Tushar Kumar Singh and Soyam Raj Kathait

Project Outcome/Deliverables

The primary deliverable is an AI-powered web application that enables users to interact with relational databases through natural-language questions rather than manually writing SQL.
The planned application includes a React/Next.js frontend, FastAPI backend, PostgreSQL integration, and AI-assisted SQL generation. Users will be able to upload datasets, submit natural-language queries, and view results through an interactive interface.
Additional deliverables include secure user authentication, RESTful API services, query validation, Redis-based task management, and a modular architecture designed for future scalability. Documentation deliverables include the Software Requirements Specification (SRS), system architecture diagrams, API documentation, and deployment guidelines.

Progress Overview

The project has progressed according to the planned development timeline. During Phase II, our team worked on finalizing the project requirements and deciding how the different components of NEXUS would work together. We selected the technologies to be used and prepared the initial structures for the frontend and backend. The PostgreSQL database schema and the main API design were also prepared. We discussed how natural-language questions could be converted into SQL queries and how the scheduler would manage the execution of different jobs.
The initial setup has been completed for several components. As mentioned in our testing status, the frontend interface, backend APIs, and database connectivity have passed their respective tests. However, connecting the AI module with the rest of the application is still in progress, and the complete system has not yet undergone final integration testing.
Our next step is to connect the individual components and make the complete workflow functional. This includes finishing the AI-based SQL generation process, implementing job scheduling, and improving database performance where required. We also need to test the application for security issues and verify that its components work together correctly before deploying the final version. These activities are planned for the final phase of the project.

Codebase Information

GitHub Repository: https://github.com/T4NIX/NEXUS
Important commits / repository milestones:

- Initial project setup and repository creation.
- Added frontend project structure and UI components.
- Implemented FastAPI backend and REST API framework.
- Designed PostgreSQL database schema and initial integration.
- Added project architecture, documentation, and configuration files.

Testing and Validation Status

Test Type	Status	Notes
Frontend UI Testing	Pass	UI functions verified.
Backend API Testing	Pass	APIs tested successfully.
Database Connectivity Testing	Pass	Database connection verified.
AI Module Integration Testing	In Progress	Integration under development.
System Integration Testing	In Progress	Final testing pending.

Deliverables Progress

Completed: SRS, problem statement, literature survey, and system architecture.
Established / in progress: Frontend interface, backend REST API development, PostgreSQL database integration, and the AI-based SQL-generation module. The core framework has been established, but full implementation and integration remain in progress.
Pending for the final phase: Complete module integration, comprehensive system testing, performance optimization, security validation, and final deployment. These activities are intended to ensure a reliable, scalable, and fully functional application.